

AWS Load-Balanced Web Server Cluster

A Self-Hosted AWS Infrastructure Project: Apache Load Balancing with AMIs, rsync & iptables

Rooble Dahir

Table of Contents

Project Overview	3
Part 1 — Golden-Image Deployment & File Synchronization	4
Part 2 — iptables Load Balancing	7
Future Enhancements	9
Conclusion.....	10

Project Overview

This project documents the design and hands-on deployment of a small load-balanced Apache web cluster on AWS EC2. Starting from a single working web server, the project builds out horizontal scaling from the ground up: capturing that server as a reusable Amazon Machine Image (AMI), deploying a second web server (www-slave1) from it, keeping content synchronized between both with SSH-key-authenticated rsync over cron, and distributing incoming HTTP traffic between them using iptables NAT rules on a router VM acting as a lightweight load balancer.

The goal was to gain practical experience with the core mechanics behind horizontal scaling and load balancing — image-based provisioning, automated content synchronization, and traffic distribution — using the same primitives (AMIs, security groups, iptables DNAT) that underpin more sophisticated production setups.

Environment

- router — NAT/firewall VM performing iptables-based load balancing, public-facing
- www — primary Apache web server (10.3.45.11), the original golden image source
- www-slave1 — AMI-cloned replica web server (10.3.45.21)
- AWS EC2, EBS, Security Groups, and AMIs (Amazon Linux 2023)

Skills Demonstrated

- AMI creation and golden-image-based horizontal scaling on AWS EC2
- SSH key-based authentication using a dedicated, task-scoped key
- rsync-based bidirectional file synchronization
- cron-based task scheduling and automation
- iptables NAT/DNAT load balancing using the statistic module
- AWS security group configuration (least-privilege port exposure)
- Linux system administration on Amazon Linux 2023 (systemd, package management)

Part 1 — Golden-Image Deployment & File Synchronization

The project began by capturing the existing, working `www` server as an AMI (`www-for-asg1-p1`), then deploying a new VM — `www-slave1` — directly from that image. Because the AMI captures the entire configured disk state, `www-slave1` came up with the same Apache installation and configuration as `www` without any manual setup.

	Name	AMI name	AMI ID	Source	Owner	Visibility	Status	Creation date
☐		www-for-asg1-p1	ami-0cf84ef2791fcad26	redacted	redacted	Private	Available	2024/10/09 16:1

Figure 1.1 — AMI captured from `www` (`www-for-asg1-p1`), used to deploy `www-slave1`

Volumes (12)											
	Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Created	Availability Zone	Volume state	Alarm si
☐	-	vol-0068c8ffca394c2a9	gp3	8 GiB	3000	125	snap-0011aac...	2024/09/30 15:41 GMT-4	us-east-1a	In-use	No alarm
☐	www-data	vol-0e340990c382cd0f5	gp3	1 GiB	3000	125	-	2024/10/02 14:55 GMT-4	us-east-1a	In-use	No alarm
☐	-	vol-0ec5882941a927e3d	gp3	1 GiB	3000	125	snap-0460d26...	2024/10/09 16:20 GMT-4	us-east-1a	In-use	No alarm
☐	-	vol-0eaa7d56a3c503f59	gp3	1 GiB	3000	125	snap-0310598...	2024/10/09 18:30 GMT-4	us-east-1a	In-use	No alarm
☐	-	vol-0ef9ceb3a5d64fc45	gp3	8 GiB	3000	125	snap-0011aac...	2024/09/18 14:28 GMT-4	us-east-1b	In-use	No alarm
☐	-	vol-08501379c5bc97cb6	gp3	8 GiB	3000	125	snap-0211189...	2024/10/09 16:20 GMT-4	us-east-1a	In-use	No alarm
☐	-	vol-0ef106d066e5dd0b8	gp3	8 GiB	3000	125	snap-0011aac...	2024/10/02 15:13 GMT-4	us-east-1a	In-use	No alarm
☐	-	vol-0f5ad12a8644212bf	gp3	1 GiB	3000	125	snap-0997390...	2024/10/09 15:20 GMT-4	us-east-1a	Available	No alarm

Figure 1.2 — EBS volumes, including the dedicated `www-data-slave1` storage volume

SSH access to `www-slave1` was enabled through the router by adding a port-forwarding rule (port 2221) and a matching `ops345routersg` security group rule, keeping each slave reachable on its own dedicated port rather than exposing SSH broadly.

DESIGN RATIONALE

Deploying from a golden image (the AMI) rather than manually reconfiguring each new server is what actually makes horizontal scaling practical: every slave starts from an identical, known-good baseline instead of accumulating manual configuration drift. This is the same principle production auto-scaling groups rely on.

Passwordless SSH & Bidirectional rsync

To keep `/var/www/html` identical across servers, a dedicated SSH key (`id_rsa_wwwsync`) was generated on `www-slave1` specifically for this synchronization task, rather than reusing a personal login key, and its public key was appended to `www`'s `authorized_keys`.

Figure 1.4 — Test files (testfile, testfile2) confirmed synced in both directions between www and www-slave1

Once verified manually, both rsync commands were added to www-slave1's crontab to run automatically every 5 minutes, keeping content synchronized without manual intervention.

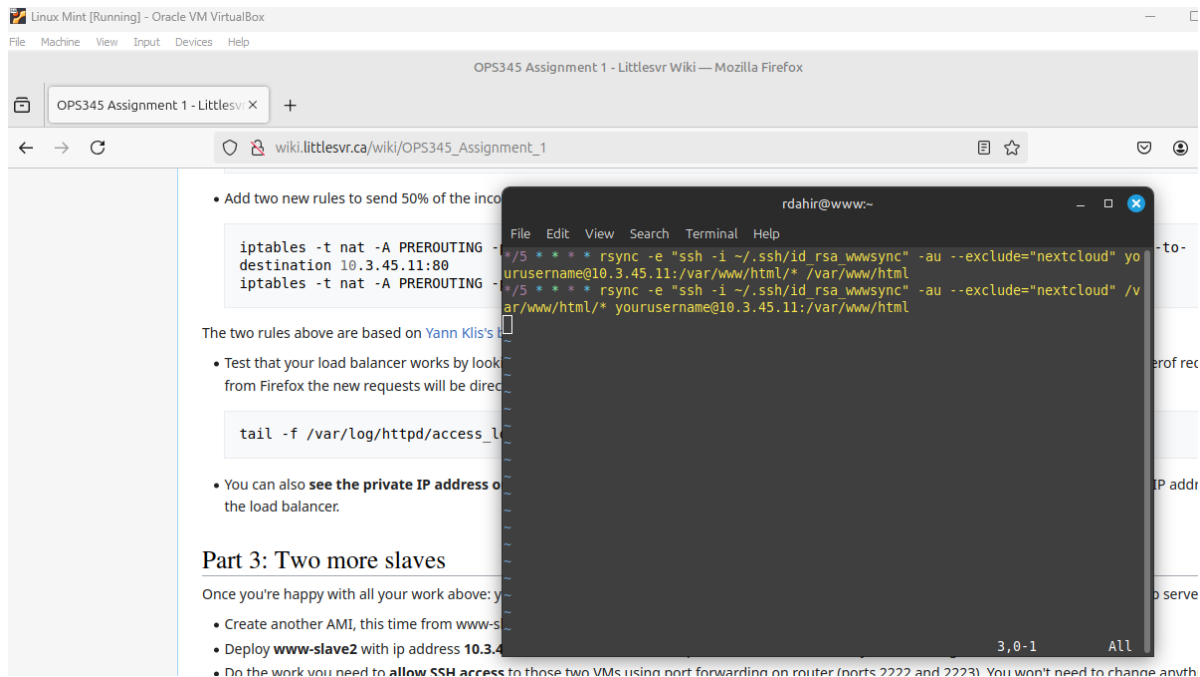


Figure 1.5 — Bidirectional rsync commands added to crontab, scheduled every 5 minutes

CHALLENGE

The scheduled rsync jobs silently failed to run at first. The cause was that cronie (the cron daemon/scheduler) isn't installed by default on Amazon Linux 2023 — `crontab -l` showed the jobs correctly, but nothing was actually executing them. Installing and enabling cronie resolved it, and the sync jobs began firing on schedule.

DESIGN RATIONALE

Using a task-scoped SSH key instead of a personal key limits the blast radius if that key is ever exposed — it can only be used for this one sync operation, not for general account access. Scheduling the sync with cron rather than running it manually is what actually makes multi-server content consistency sustainable; a manually-run sync is only as reliable as remembering to run it.

Part 2 — iptables Load Balancing

With two web servers serving identical content, the router's iptables NAT table was modified to distribute incoming port 80 traffic between them. Before making changes, the existing working iptables rules were backed up (cp /etc/sysconfig/iptables /root/iptables-before-asg1) as a safety net.

The original single-destination port-80 rule was replaced with two PREROUTING DNAT rules: the first uses the statistic module to randomly match roughly 50% of incoming connections and forward them to www, and the second catches everything else and forwards it to www-slave1.

```
iptables -t nat -A PREROUTING -p tcp -m tcp --dport 80 \
-m statistic --mode random --probability 0.5 \
-j DNAT --to-destination 10.3.45.11:80

iptables -t nat -A PREROUTING -p tcp -m tcp --dport 80 \
-j DNAT --to-destination 10.3.45.21:80
```

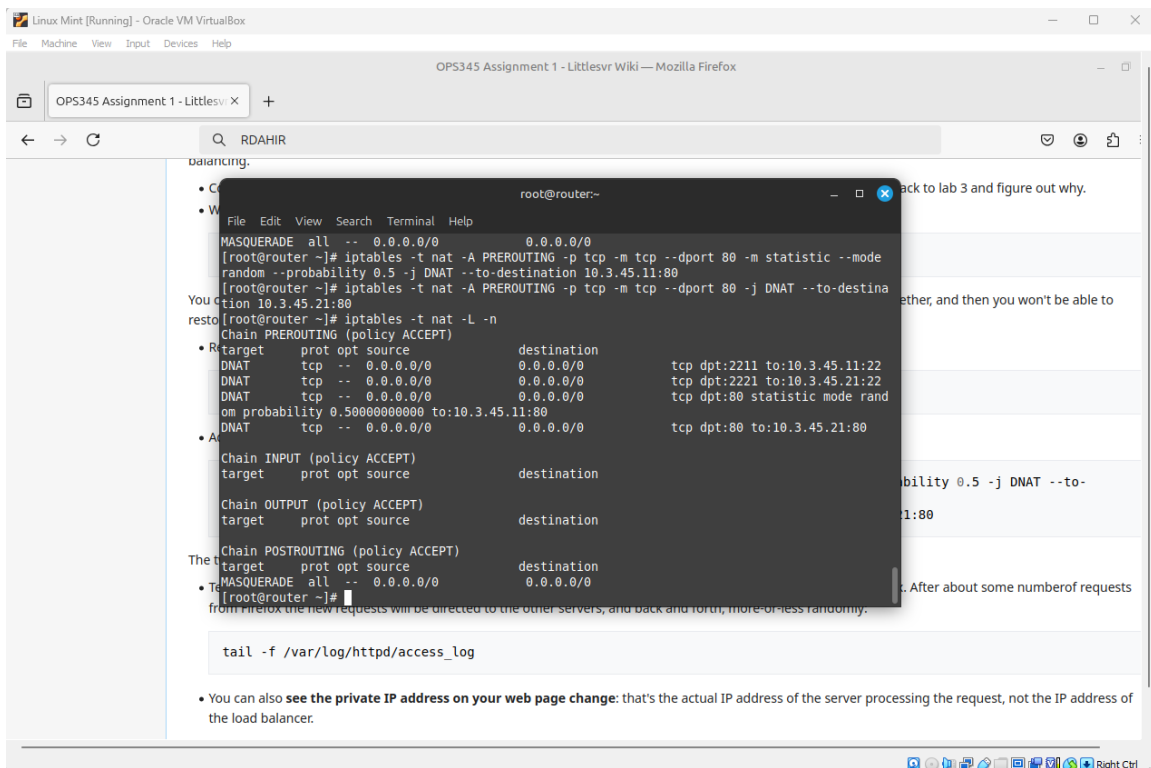


Figure 2.1 — iptables NAT table on router, showing the SSH port-forwarding rules (2211/2221) and the 50/50 DNAT split for port 80

Traffic reaching the servers was confirmed by tailing the Apache access log directly on www while requests were made through the router — confirming the router's forwarding chain was actually delivering HTTP traffic end-to-end rather than just looking correct in the ruleset.

Future Enhancements

The following upgrades were not implemented in this lab environment but represent the logical next steps toward a production-ready deployment:

- Scale beyond two servers by branching a new AMI from `www-slave1` (inheriting its sync configuration) and adding additional weighted iptables rules — e.g. probabilities of 0.25, 0.33, and 0.5 — so each added server receives an equal share of traffic.
- Replace probability-based iptables DNAT with a real load balancer (HAProxy or nginx) that performs active health checks, so traffic automatically stops routing to a downed server.
- Replace manual bidirectional rsync with a shared backend (EFS or S3) to remove sync lag and eliminate the possibility of write conflicts between servers.
- Automate AMI creation, slave deployment, and security group changes with Infrastructure-as-Code (Terraform or CloudFormation) instead of manual console steps.
- Add centralized logging and monitoring across all servers instead of tailing individual access logs by hand.
- Add HTTPS/TLS termination at the load balancer rather than serving plaintext HTTP.

Conclusion

This project delivered a working, horizontally-scaled Apache web cluster on AWS: a golden-image deployment workflow for provisioning identical servers, automated bidirectional content synchronization, and iptables-based traffic distribution across the resulting pool.

Together, these stages reflect core skills behind horizontal scaling and load balancing — image-based provisioning, automated synchronization, and traffic distribution — the same underlying primitives that more sophisticated production load balancers and orchestration tools build on.